エヂンズゲート

科学技術計算

第1回：オリエンテーション

注意事項

- 高度2年の方
  - この講義は演習を含むため、メディア情報分野以外の学生は，この受講しても単位として認められません。自分野向けに開講される講義を受けて下さい。
- 創造2年の方
  - 主分野によって以下のように受講すること
    「メディア」→ 坂上の講義へ
    「知能」「ネットワーク」→ 玉木先生の講義へ
- 1年，3年の方
  - 講義が重なる場合の措置は先生・教務学生委員へ相談

科学技術計算

- 数値解析（Numerical Analysis）を主に扱う

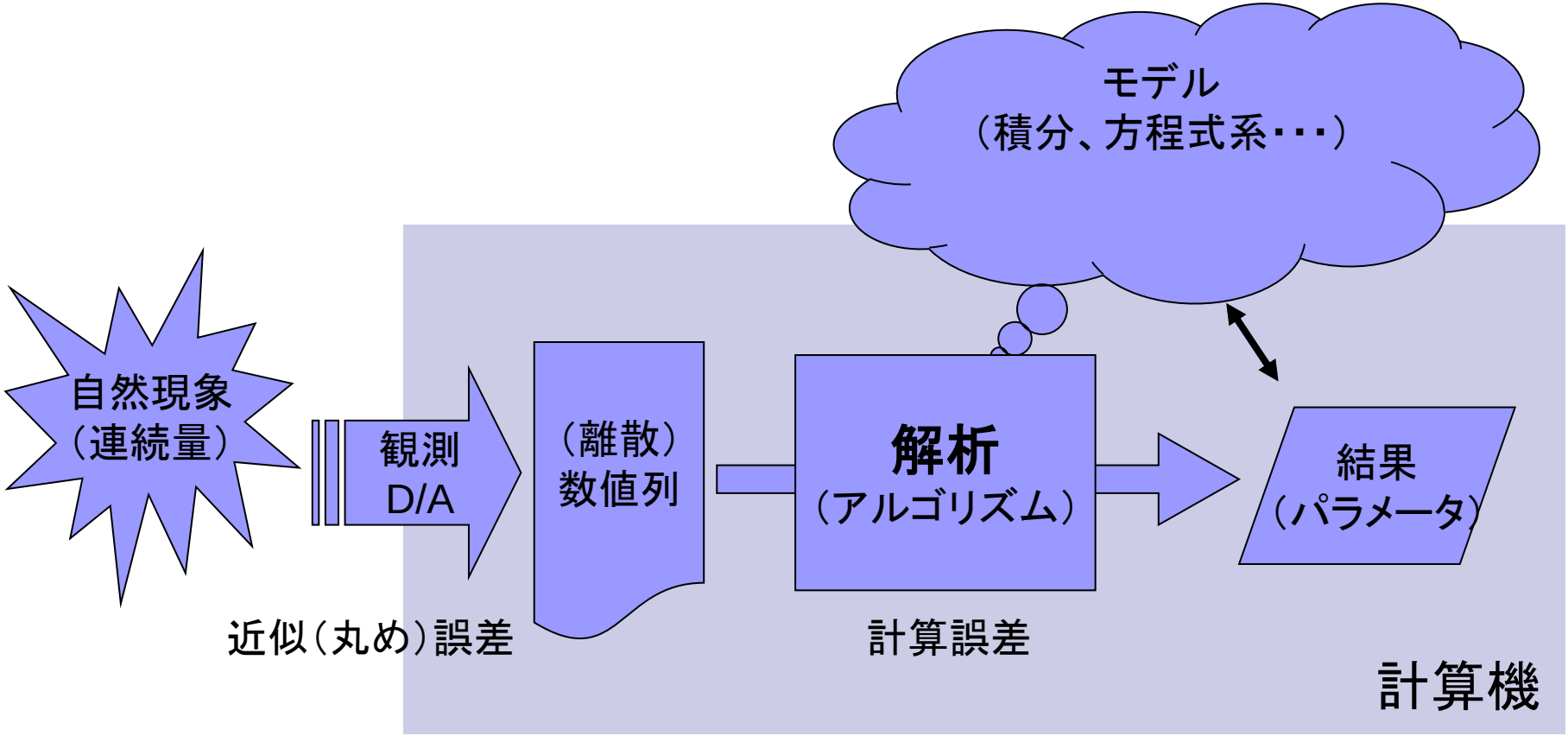
  解析的に解くことが不可能な数学の問題を、
  種々の数値計算アルゴリズムとコンピュータを
  利用して**数値的に求める**手法に関する学問

数学的手法 ⟺ 数式的手法

解析的手法　　数値的手法

・より大規模な問題　⇔　計算機の発達
・実世界を対象とするより複雑な問題（流体力学、熱解析、etc..）

何のため？



現象のモデル化
データの予測

応用例

- 様々な系の解析
  - 画像解析・音響解析
- 自動モデル化
  - 機械学習・ニューラルネットワーク
- 予測
  - 需要予測
  - スケジューリング

データに基づく分析・解析・モデル化

「計算機で数値的に解く」とは

- 例：整数の立方根を求めよ

```
x = int(input("Enter an integer: ")
ans = 0
while ans**3 < abs(x):
    ans = ans + 1
if ans**3 != abs(x):
    print(x, "is not a perfect cube")
else:
    if x < 0:
        ans = -ans
    print("Cube root of", x, "is ", ans)
```

「計算機で数値的に解く」とは

- 数値的解法の1例：総当たり法
  - 正解を得るか候補が無くなるまですべて列挙
- 数学っぽくない？効率悪い？
  - 多くの場合、総当たり法は最も実用的な解法
  - 実装が簡単、理解するのが楽
  - 計算量は多くの場合無問題
    - たいていの問題は一瞬あるいは現実的な時間で処理
    - （ただし深層学習等の超大規模計算では解法の質も重要）
- 計算機ならではの解き方
  - ただし計算機ならではの注意すべき特性があり

講義の目的と狙い

計算機による数値計算では、計算機特有の考慮すべき特性が存在する。

また、計算機ならではの様々な技法が存在する

計算機内部での数値計算の仕組みを理解し、様々な数値計算法の原理を学び、その仕組みを演習を通じて体得する。

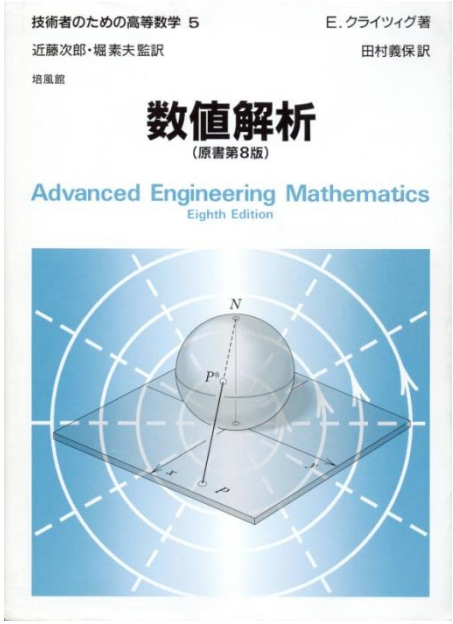
計算機上でのプログラミングによる演習

講義資料

- スライド資料（Moodleで配布）
- 教科書

技術者のための高等数学5
「数値解析（原書第8版）」
E.クライツィグ著 田村義保訳(培風館)

プログラムに近い実践的・具体的な解説が特徴
ただし受講において<u>購入は不要</u>



講義形態（予定）

- ２週間で1セット、６～７セット
  - 第１週：講義＋演習
  - 第２週：演習＋レポート作成
- 演習ごとにレポートを提出
  - moodle で提出
  - 提出期限：それぞれ終わった1週間後
  - 最終期限：学期末

単位取得要件（1）

- 成績は演習内容および提出物で決まる

- 提出物の評価基準
  - 提出されたプログラム
  - 課題の理解度
  - 考察
  - その他

単位取得要件（2）

- 全て出席することが前提
  - 全ての回で出席をカウント
  - 無断欠席、レポートの提出遅れは大幅減点
- 特に、以下の場合は無条件で不合格
  - 最終締切時点で未提出のレポートがある場合
  - 3回以上レポートの提出が遅れた場合

講義室について

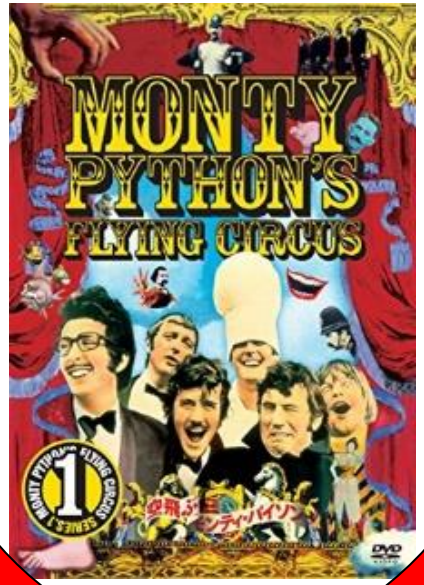
- １回目講義：講義室に集合，講義の後演習
- ２回目講義：演習

講義予定

① オリエンテーション
② Python 導入
③ 計算誤差
④ 連立一次方程式
⑤ 関数近似（線形回帰）
⑥ 補間
⑦ 固有値問題・主成分分析

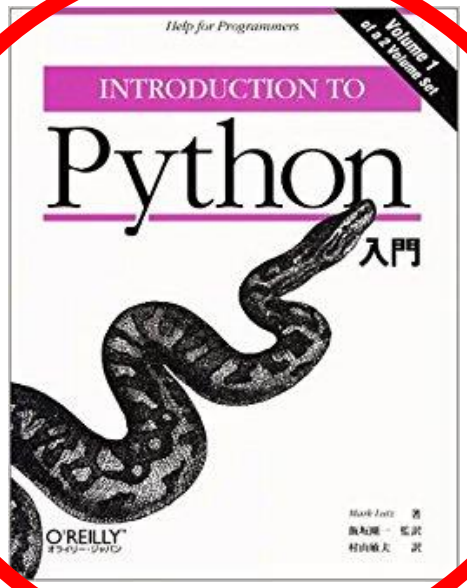
パイソン？

キリスの
コメディグループ



M thon's Flyin s
(1969

拳銃（コルト・パイソン）



グイド・ヴァンロッサム（1956～）

Python の発案者

Pythonの起源について、ヴァンロッサム自身は1996年に次のように書いている。

> Over six years ago, in December 1989, I was looking for a "hobby" programming project that would keep me occupied during the week around Christmas. My office … would be closed, but I had a home computer, and not much else on my hands. I decided to write an interpreter for the new scripting language I had been thinking about lately: a descendant of ABC that would appeal to Unix/C hackers. I chose Python as a working title for the project, being in a slightly irreverent mood (and a big fan of Monty Python's Flying Circus).[6]
>
> — *Guido van Rossum、Foreword for "Programming Python"*

> （日本語訳）6年以上前の1989年12月、私はクリスマス前後の週の暇つぶしのため「趣味」のプログラミングプロジェクトを探していた。オフィスは閉まっているが、自宅にはホームコンピュータがあるし、他にすることがなかった。私は最近考えていた新しいスクリプト言語のインタプリタを書くことにした。それは、ABCからの派生であり、Unix/Cハッカーの注意をひきつけるかもしれないと考えた。ちょっとしたいたずら心から（『空飛ぶモンティ・パイソン』の熱烈なファンだったというのも理由の1つ）、プロジェクトの仮称をPythonにした[6]
>
> ―グイド・ヴァンロッサム、「*Programming Python*」の序文

https://ja.Wikipedia.org/wiki/グイド・ヴァンロッサム

Top Ten Programming Languages by IEEE *Spectrum*

IEEE：米国電気電子学会

2017

Language Rank	Types	Spectrum Ranking
1. Python		100.0
2. C		99.7
3. Java		99.5
4. C++		97.1
5. C#		87.7
6. R		87.7
7. JavaScript		85.6
8. PHP		81.2
9. Go		75.1
10. Swift		73.7

2019

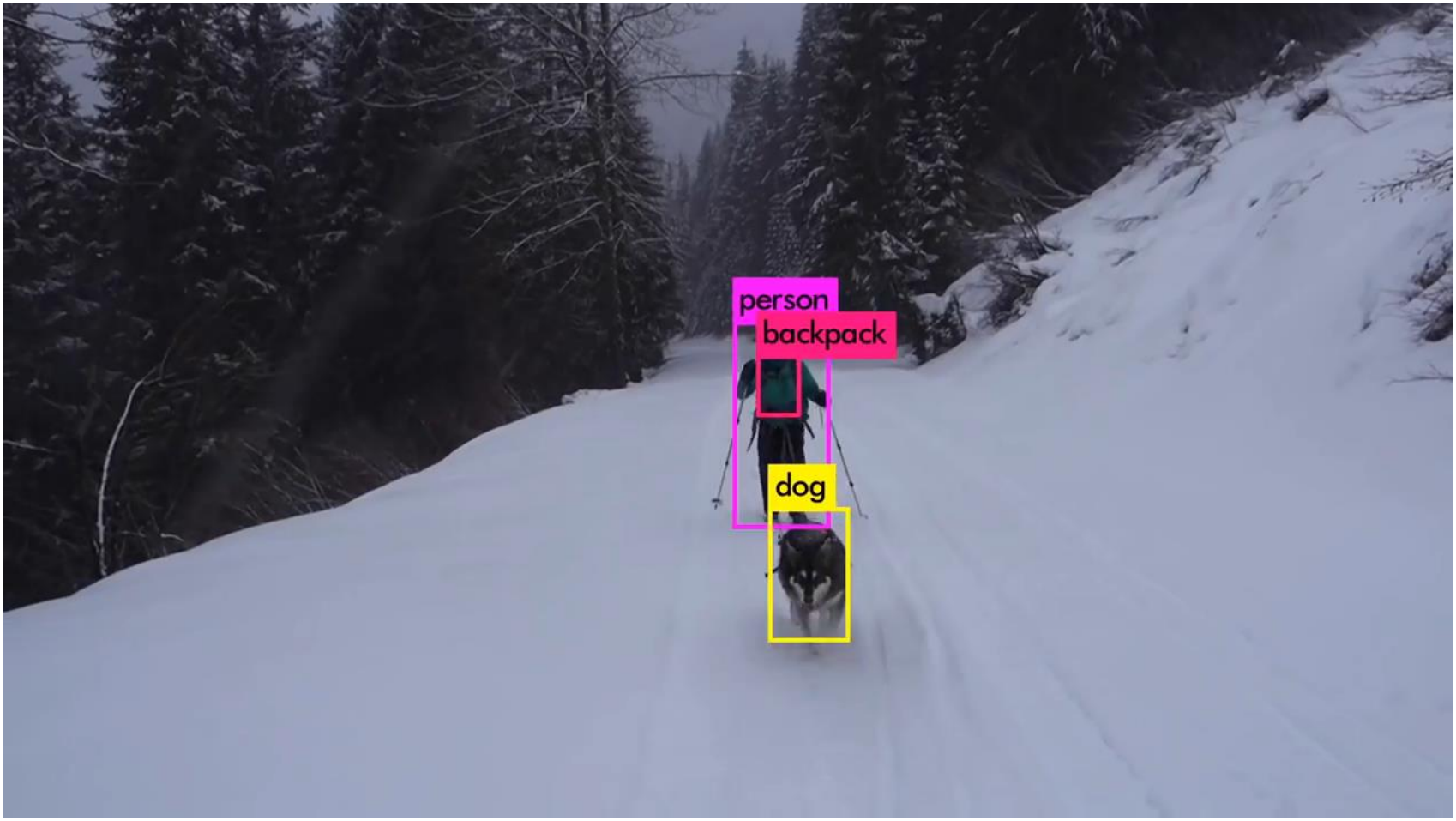
Rank	Language	Type	Score
1	Python		100.0
2	Java		96.3
3	C		94.4
4	C++		87.5
5	R		81.5
6	JavaScript		79.4
7	C#		74.5
8	Matlab		70.6
9	Swift		69.1
10	Go		68.0

https://spectrum.ieee.org/computing/software/the-top-programming-languages-2019

なぜ Python？

- 「わかりやすさ」重視の設計
- インタープリタで直感的に試せる
- 超多数のすぐに使える最新モジュール
  - どんどん import
  - 空だって飛べる（← 有名な言葉らしい）
- 開発効率が良い
  - 気の利いた言語
  - 適当に書いてもだいたい動く
  - 丁寧なエラーメッセージ
- **数値計算がやりやすい**
  - リストや配列の扱いが柔軟で簡単
  - 高速・便利な数値計算ライブラリの充実（→機械学習）

Pythonでできること



YOLOv3 ワシントン大学

- https://twitter.com/pess_r/status/1557517982095626241

向き・不向き

- Pythonが最適であるもの
  - ラピッドプロトタイピング
  - データの分析
  - 数値計算・機械学習のプログラム
  - とりあえず作ってみるプログラム
- Pythonがかならずしも最適ではないもの
  - 型等のチェックは（厳密な言語に比べると）脆弱
  - 高い信頼性が求められるプログラム
  - 多くの人が関わるプログラム
  - 長期にわたり開発・維持するプログラム
  - 少ないリソースで高速な動作が期待されるプログラム
  - モバイルプログラミング

Python2 と Python3

本演習は Python3 ベースで行います

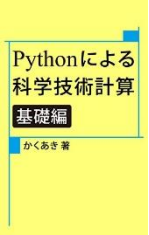
- 1990年に Guido von Rossum が開発
- 2000年の Python2.0 でメジャーへ
  - 大幅な言語仕様の拡張および改善
  - 多くの人々によるPython用ライブラリの開発
- 2008年にPython3.0がリリース
  - Python2 の多くの矛盾を解決
  - Python2 と互換性なし！
- ここ数年で Python 3 がほぼメジャーに
  - 重要なライブラリのほとんどがPython3へ移植された
  - 特に 3.5 以降は安定

調べるとき、情報が ver. 2 向けの古いものでないか気をつけること

Pythonの勉強

講義と並行して自学していくことを推奨します

- 参考書（たくさんの参考書があります　下記は例）
  - Pythonによる科学技術計算（基礎編・実践編）
    - 超基礎から簡単な数値計算の実行まで
    - とりあえず動かしながら雰囲気を掴める



  - みんなのPython
    - Python入門書の定番
    - 言語仕様からしっかり学べる

  - MIT Python言語によるプログラミングイントロダクション第2版 – データサイエンスとアプリケーション -
    - 研究者志向の学生向け。
    - 研究に直結していて読み応えあり。でも高い。

- オンライン学習
  - Codeacademy, CodingBat（Pythonのみ）等

次回から

- 実際の演習を開始します
- 開発環境の整備については別資料
- 講義資料
  - 講義開始前に Moodle に掲示する
  - 紙への印刷は不要